

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**ДЕМОН ЗА МАРШРУТИЗАЦИЯ ПО
СЪСТОЯНИЕ НА ВРЪЗКИТЕ И
ДИСКРЕТНО-СЪБИТИЕН СИМУЛАТОР ЗА
ИЗСЛЕДВАНЕ НА СХОДИМОСТТА В ТСП/Р
МРЕЖИ**

КУРСОВ ПРОЕКТ

по дисциплина „Мобилни и стационарни комуникационни мрежи“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Доц. д-р инж. Георги Цочев, Доц. д-р инж. Георги Найденов, Доц. д-р Петко Стоянов

София, **[TODO: година на предаване]**

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД .	3
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	5
2.1. Семейство на протокола	5
2.2. Модел на времето	5
2.3. Език и инструментариум за изграждане	5
2.4. Еталон за сверяване на коректността	6
2.5. Обхват спрямо учебното съдържание	6
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	7
3.1. Обща структура	7
3.2. Абстракция върху времето и входа и изхода	7
3.3. Протоколно ядро	8
3.4. Дискретно-събитиеен симулатор	9
3.5. Записване на събития за анализ	10
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	11
4.1. Организация на изходния текст	11
4.2. Формат по мрежата	11
4.3. Автомат за състоянията на съседите	12
4.4. Синхронизиране на базите	13
4.5. Разпространение на обявленията	13
4.6. Изчисляване и инсталиране на маршрути	14
4.7. Симулатор и внасяне на откази	15
4.8. Жив режим	15
4.9. Съвместимост с еталонна реализация	16
V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА	17
5.1. Измервани величини	17
5.2. Конфигурация на машината	17

5.3. Топологии	18
5.4. Управлявани параметри	18
5.5. Повторения и условия за валидност	19
5.6. Форма на резултатите	20
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	21
6.1. Проведени измервания	21
6.2. Време за сходимост спрямо размера на мрежата	21
6.3. Време за сходимост спрямо таймерите	22
6.4. Натоварване от лавинното разпространение спрямо плътността	23
6.5. Преходни цикли спрямо вида на отказа	24
6.6. Работен пример: пълна времева картина	25
6.7. Възпроизводимост	26
6.8. Сверяване с еталонна реализация	26
6.9. Обсъждане	27
ЗАКЛЮЧЕНИЕ	28
ЛИТЕРАТУРА	31
ПРИЛОЖЕНИЯ	32

УВОД

Маршрутизацията в мрежите по стека TCP/IP се описва в учебната литература като решена задача: всеки маршрутизатор поддържа представа за топологията, изчислява най-кратките пътища и попълва маршрутна таблица. Поведението, което определя качеството на мрежата на практика, обаче не е в установеното състояние, а в прехода между две установени състояния. Когато една връзка откаже, различните възли научават за отказа в различен момент и за краткия интервал, докато представите им се разминават, мрежата може да образува преходни цикли и да губи пакети по пътища, които изглеждат валидни за всеки възел поотделно. Този интервал се нарича време за сходимост и е основният предмет на настоящия курсов проект.

Причината преходът да се изучава рядко в лабораторни условия е практическа. Отказът на връзка в реална мрежа настъпва в момент, който не се повтаря, а таймерите на протокола се задвижват от истински часовник. Две последователни наблюдения на едно и също събитие дават различни числа и разликата между тях не може да бъде отдадена на промяна в протокола. За да бъде измерим, преходът трябва да бъде възпроизводим, а възпроизводимостта изисква времето да бъде управлявано от програмата, а не от часовника на машината.

Целта на проекта е да се изгради демон за маршрутизация по състояние на връзките от семейството OSPF заедно с дискретно-събитиеен симулатор, който изпълнява същия програмен код върху управлявано време, и с този инструмент да се измери времето за сходимост и образуването на преходни цикли при отказ на връзка.

За постигането на целта са формулирани следните **задачи**:

1. литературен преглед на принципите на маршрутизация и управление на потоците в мрежи по TCP/IP, поставен в контекста на дисциплината;
2. анализ на възможните решения по езика, модела на времето, транспортния слой и еталонната реализация, и обосновка на направения избор;
3. проектиране на демона така, че протоколната логика да не зависи от източника на време и от източника на пакети;
4. програмна реализация на откриване на съседи, разпространение на обявленията за състояние на връзките, изчисляване на най-кратките пътища и инсталиране на маршрути;

5. изграждане на методика за възпроизводимо измерване на сходимостта при отказ на връзка;
6. провеждане на измерванията и анализ на получените резултати.

Обхватът на реализацията е маршрутизиращият слой на стека TCP/IP. Безжичните, спътниковите и клетъчните мрежи, както и ATM, присъстват в раздел I като контекст, защото са част от учебното съдържание на дисциплината, но не влизат в реализацията. Причината е методологична, а не липса на интерес: тези среди въвеждат модел на канала със загуби и променлив капацитет, чиято достоверност сама по себе си е отделна изследователска задача, и включването им би направило измерената сходимост зависима от достоверността на модела на канала вместо от поведението на протокола.

Структура на изложението. Раздел I представя предметната област. Раздел II излага разгледаните алтернативи и обосновава избора. Раздел III описва проектирането, а раздел IV програмната реализация. Раздел V описва методиката за измерване, а раздел VI получените резултати. Заключение обобщава изводите и очертава посоките за по-нататъшна работа.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

Дисциплината „Мобилни и стационарни комуникационни мрежи“ обхваща два слоя, които се изучават заедно, но се проектират по различни принципи. Първият е преносната среда със своите разновидности: стационарни кабелни мрежи, безжични мрежи с локален обхват, клетъчни мрежи с преминаване между клетки, спътникови връзки с голямо закъснение на разпространение и мрежи с виртуални канали от типа ATM. Вторият е мрежовият слой на стека TCP/IP, който работи над всяка от тези среди и трябва да остава функционален, когато характеристиките им се променят. Учебното съдържание отделя специално внимание на принципите на маршрутизация и управление на потоците в TCP/IP базирани мрежи и именно този слой е предмет на настоящия проект.

Класификацията на протоколите за маршрутизация вътре в една автономна система е установена и се свежда до две семейства. Протоколите по вектор на разстоянието обменят със съседите си собствената си маршрутна таблица, като RIP [1]. Те са прости за реализация, но страдат от бавно разпространение на лошите новини и изискват допълнителни механизми срещу образуването на цикли. Протоколите по състояние на връзките разпространяват описание на локалната свързаност до всички възли, така че всеки възел изгражда еднакъв граф на топологията и изчислява маршрутите самостоятелно. OSPF версия 2 [2] е каноничното описание на този подход, а версия 3 го пренася върху IPv6 [3]. Сравнителен анализ на двете семейства, включително причините за преходните цикли, е даден при Perlman [4].

Изчисляването на маршрутите при протоколите по състояние на връзките се свежда до търсене на дърво от най-кратки пътища с корен текущия възел, тоест до алгоритъма на Дейкстра [5]. Изборът на алгоритъм е безспорен, но времето за сходимост не се определя от него: изчислението е бързо в сравнение с интервалите, през които възлите изобщо научават за промяната. Тези интервали се задават от таймерите за откриване на съседи и от закъснението на лавинното разпространение на обявленията. Затова литературата за ускоряване на сходимостта се съсредоточава върху по-бързо откриване на отказа, например чрез двупосочно откриване на препращането (BFD) [6], а не върху оптимизация на самото изчисление.

Управлението на потоците в TCP/IP се разполага в друг слой, но е свързано със

сходимостта по наблюдаем начин. Прозорецът за претоварване на TCP се свива при загуба на пакети [7, 8], а точно преходните цикли при пренастройване на маршрутите произвеждат загуба, която за подателя е неразличима от загуба поради претоварване. Реакцията на транспортния слой при промяна в маршрутизацията е следствие от поведението на мрежовия слой и това прави времето за сходимост величина с пряк ефект върху пропускателната способност, а не вътрешна характеристика на протокола. Общата рамка на двата слоя е изложена при Kurose и Ross [9], а нормативното описание на TCP е в [10].

Инструментите за изследване на такива преходи също са установени. Симулаторите с дискретни събития OMNeT++ [11] и ns-3 [12] предоставят зряла среда с модели на среди и протоколи. Те обаче симулират собствени модели на протокола, докато настоящият проект се стреми към друго свойство: една и съща реализация да бъде изпълнима както над симулирано, така и над реално време. **[TODO: да се добави преглед на публикации, измерващи сходимостта на OSPF в реални мрежи, след като бъде избран конкретен набор]**

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

2.1. Семейство на протокола

Първият избор е между двете семейства протоколи за вътрешна маршрутизация. Протокол по вектор на разстоянието би бил по-кратък за реализация, но при него всеки възел вижда само обобщени разстояния и няма представа за топологията. Измерването на преходни цикли тогава изисква външен наблюдател, който събира таблиците на всички възли и възстановява глобалната картина. При протокол по състояние на връзките всеки възел вече притежава описание на топологията, така че разминаването между представите на два възела е непосредствено наблюдаема величина. Избран е подходът по състояние на връзките именно защото прави измерваната величина видима отвътре.

2.2. Модел на времето

Вторият избор е между три модела на времето. Работа изцяло върху реални интерфейси дава най-висока достоверност, но не дава повтораемост: моментът на отказа не се възпроизвежда и разсейването на резултатите скрива ефекта, който се изследва. Работа изцяло в симулатор дава пълна повтораемост, но проверява модела, а не програмата, която после ще работи в мрежа. Приет е трети вариант: протоколната логика не получава достъп до часовник и до сокети, а взаимодейства с абстракция, зад която стои или дискретно събитийна опашка с виртуален часовник, или системният часовник и реалните сокети. Едни и същи изходни файлове на протокола се компилират в двата режима. Цената на този избор е допълнителна абстракция и невъзможност да се използва пряко системно извикване в протоколния код. Ползата е, че измереното поведение принадлежи на реализацията, а не на нейн симулационен двойник.

2.3. Език и инструментариум за изграждане

Разгледани са C++, Rust и Go. Go предлага най-кратък път до работеща мрежова програма, но неговата среда за изпълнение управлява планирането и събирането на паметта, което внася закъснения, невъзможни за отстраняване от измерването. Rust дава силни гаранции за безопасност на паметта, но еталонните реализации, спрямо които се

сверява коректността, са на C и интеграцията с тях изисква допълнителен слой. Избран е C++ по стандарта от 2020 година [13]: детерминирано управление на ресурсите, пряка съвместимост с интерфейсите на операционната система и наличие на конструкции, които правят абстракцията върху времето безплатна по време на изпълнение. За изграждане е избран CMake, защото трябва да произвежда еднакъв резултат върху Windows, Linux и macOS.

2.4. Еталон за сверяване на коректността

Като еталон са разгледани FRRouting и BIRD. И двата са зрели реализации на OSPF, които работят срещу оборудване на производители и следователно тълкуват [2] по начин, приет от практиката. Еталонът би се използвал за проверка на коректността, тоест за съвместимост на форматите на пакетите и на съдържанието на изчислените маршрути, а не за сравнение по бързина. Сравнение по бързина би било подвеждащо, защото обхватът на реализацията в този проект е подмножество от този на еталона.

Този избор беше преразгледан по време на реализацията и **отхвърлен**. Съвместимостта с външна реализация изисква форматът по мрежата да бъде този на OSPFv2 по [2], включително двойките пакети за обзор на базата с водещ и подчинен, областите и типовете обявления, които настоящият обхват не използва. Реализацията на кодирането е съизмерима по обем с останалата част от проекта и не допринася за измерваната величина, защото времето за сходимост не зависи от подредбата на полетата в пакета. Вместо външен еталон е използван вътрешен: маршрутните таблици на всички възли се сравняват с таблици, изчислени независимо върху истинската топология. Разликата в силата на двете проверки е отчетена в раздел VI, а не премълчана.

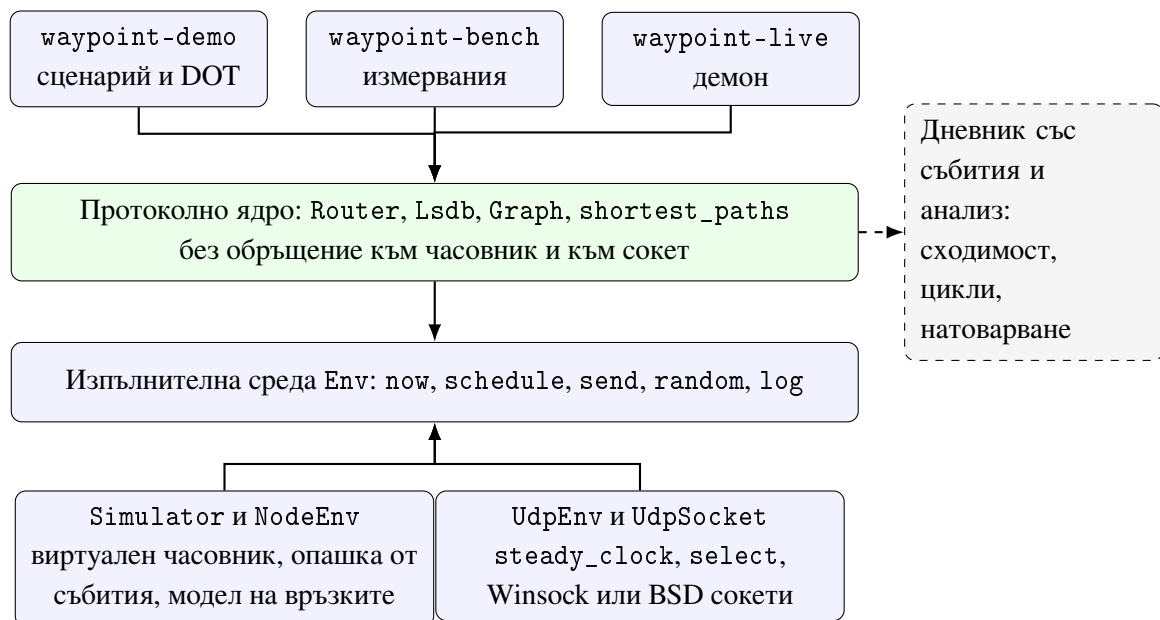
2.5. Обхват спрямо учебното съдържание

Дисциплината обхваща и безжични, клетъчни, спътникови и АТМ мрежи. Те са включени в раздел I като контекст, но не в реализацията. Моделирането на канал със загуби и променливо закъснение изисква обосноваване на самия модел и измерената сходимост тогава зависи от достоверността на този модел. Проектът избира по-тесен обхват с ясно измерима величина пред по-широк обхват с недоказуеми числа.

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

3.1. Обща структура

Системата е разделена на четири слоя (Фиг. 1). Най-долу стои изпълнителна среда, която предоставя време, планиране на таймери, изпращане на съобщения и източник на псевдослучайни числа. Над нея стои протоколното ядро, което съдържа автомата за състоянията на съседите, базата с обявленията за състояние на връзките и изчисляването на маршрутите. Над ядрото стои приложната обвивка: три изпълними програми, които свързват ядрото с конкретна изпълнителна среда. Отстрани стои слой за анализ, който работи само върху маршрутните таблици и върху дневника със събития.



Фигура 1. Слоевете на системата. Стрелките сочат посоката на зависимостта.

Протоколното ядро не съдържа обръщения към часовника и към сокетите. Границата е структурна: заглавните файлове на платформата се включват единствено в `src/udp.c`, така че нарушение на границата се вижда като нов такъв ред в друг файл и се открива с търсене в изходния текст. Двата режима изпълняват едни и същи обектни файлове на ядрото.

3.2. Абстракция върху времето и входа и изхода

Границата е интерфейсът `Env` с пет операции. `now` връща текущия момент в микросекунди. `schedule` планира еднократно извикване след зададено закъснение. `send`

предава готова дейтаграма към съсед. `random` връща псевдослучайно число, използвано за разместване на началните моменти на таймерите. `log` записва събитие в дневника.

Интерфейсът съзнателно няма операция за отмяна на таймер. Отмяната би изисквала идентификатор на таймера, срок на валидност на този идентификатор и правило какво се случва при отмяна на вече изпълняващ се таймер. Вместо това всеки таймер носи *печат на поколението*: полето, което таймерът пази, се сравнява при изпълнение със стойността в структурата на съседа, и при несъвпадение извикването се връща без действие. Един целочислен инкремент отменя произволен брой висящи таймери. Цената е, че отменените таймери остават в опашката до момента си, което при виртуално време не струва нищо.

В режим на симулация източникът на време е виртуален часовник, който се движи само когато опашката извади следващото събитие. Между две събития не изтича време, така че изпълнението не зависи от натовареността на машината. В режим на демон източникът е монотонният часовник на операционната система, а изпращането е `sendto` по UDP.

Механизмът за изчакване на събития в режим на демон е `select` и на трите целеви платформи. Разгледани бяха `epoll` на Linux, `kqueue` на macOS и `WSAPoll` на Windows, но всеки от тях изисква отделна реализация, а предимството им се проявява при стотици дескриптори. Един демон следи един сокет, така че разликата не може да повлияе на измерваната величина. Цикълът изчислява времето до следващия таймер, изчаква толкова на сокета и след това изпълнява всички дозрели таймери. Единствените разлики между платформите са типът на манипулатора, извикването за затваряне и начинът на съобщаване на грешка; те са събрани в `src/udp.c` и не се виждат никъде другаде.

3.3. Протоколно ядро

Ядрото поддържа три структури. Първата е таблица със съседите, всеки от които се намира в състояние от автомата за съседство и има таймери за поддържане и за отпадане. Втората е база с обявленията за състояние на връзките, индексирани по подател и с поредни номера, които определят кое обявление е по-ново. Третата е маршрутната таблица, получена от изчисляването на дървото от най-кратки пътища върху графа, реконструиран от базата. Разпространението на обявленията е лавинно: получено по-ново обявление се записва и се препраща по всички връзки без тази, по която е дошло.

Графът, реконструиран от базата, е *насочен*, защото всеки маршрутизатор обявява само своята страна на всяка връзка. Преди изчислението се взема двупосочното му подмножество: ребро участва само ако и двата му края са го обявили. Това правило е причината едностранният отказ на връзка да води до правилно пренастройване, а не до постоянна черна дупка.

Разделението между база и маршрутна таблица е съществено за измерването. Моментът, в който възел научава за промяна, моментът, в който изчислява нови маршрути, и моментът, в който новата таблица влиза в сила, са три различни момента. Изчисляването и влизането в сила са отделни стъпки в реализацията именно защото интервалът между тях е мястото, където възникват преходните цикли: възелът вече знае новата топология, но още препраща по старата.

3.4. Дискретно-събитиеен симулатор

Симулаторът съдържа опашка, подредена по двойката (момент, пореден номер на вписване). Поредният номер е съществен: без него две събития с еднакъв момент биха се изпълнявали в ред, зависещ от вътрешното устройство на контейнера, и изпълнението не би било възпроизводимо. Всяко изваждане от опашката премества виртуалния часовник напред.

Всяка връзка се моделира като две независими посоки. Посоката има закъснение на разпространение, капацитет и момент, до който е заета. Пакет, предложен докато предишният още се изпраща, се нарежда след него, така че моделът включва и опашка, и сериализация, не само закъснение. Отказите се планират като събития във виртуално време и биват четири вида: отказ на връзка в двете посоки, отказ само в едната посока, отказ на възел и възстановяване. Отказът на възел се моделира като пълна изолация, защото за останалата част от мрежата спрял възел и възел, чиито връзки са отпаднали, са едно и също наблюдение.

Източникът на псевдослучайни числа е `std::mt19937_64` с явно зададено зърно. Стандартните разпределения не се използват, защото те не са преносими между реализации на стандартната библиотека; вместо тях се взема остатък по модул, чиято аритметика е еднаква навсякъде. Изборът е между лека неравномерност и възпроизводимост между платформи, и е направен в полза на второто.

3.5. Записване на събития за анализ

Всяко значимо събитие се записва като запис от шест полета: момент, възел, вид, отсрещен възел и две числови полета, чието значение зависи от вида. Видовете са промяна на състоянието на съсед, породено обявление, прието обявление, препратено обявление, изчислени маршрути, инсталирани маршрути и четирите вида внесен отказ. Форматът е даден в приложение А.

Анализът след изпълнението работи само върху маршрутните таблици и върху този дневник, и не се нуждае от достъп до вътрешното състояние. Сходимостта се проверява чрез сравнение на маршрутната таблица на всеки възел с таблицата, изчислена независимо върху истинската топология. Преходните цикли се откриват чрез проследяване на следващите стъпки за всяка двойка източник и цел: ако проследяването посети възел два пъти, двойката е в цикъл. Двете проверки се извикват след всяко инсталиране на маршрути, така че моментът на достигане на сходимост и границите на всеки цикъл се засичат с точността на събитието, а не на извадка.

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

4.1. Организация на изходния текст

Изходният текст е разделен на библиотека и три изпълними програми. Библиотеката съдържа единадесет заглавни файла в `include/waypoint/` и девет файла с реализация в `src/`: `types` за скаларните типове, `packet` за формата по мрежата, `lsdb` за базата, `spf` за графа и изчислението, `neighbor` за автомата, `router` за протоколното ядро, `topology` за топологиите, `sim` за симулатора, `analysis` за анализа и `udp` за живия режим; `types` и `env` нямат файл с реализация, защото съдържат само декларации. Програмите в `apps/` са `waypoint-demo` със сценария на демонстрацията, `waypoint-bench` с измерванията и `waypoint-live` с демона. Общият обем е 4776 реда: 2921 реда библиотека, 718 реда програми и 1137 реда тестове.

Изграждането е с CMake по стандарт C++20 [13]. **Проектът няма зависимости извън стандартната библиотека и системните заглавни файлове за сокети.** Това не е предпочитание по стил: изграждане, което изисква мениджър на пакети или изтегляне на код по мрежата при конфигуриране, не се възпроизвежда на чужда машина без допълнителна подготовка, а възпроизводимостта е предмет на този проект. Затова средството за тестване е собствено, 132 реда макроси с проверки в `tests/check.hpp` и `tests/check.cpp`, регистрирани в CTest, а измерването на време ползва пряко `std::chrono::steady_clock`. Първоначалният план предвиждаше GoogleTest; той беше отхвърлен по същата причина.

4.2. Формат по мрежата

Всички цели числа се кодират явно, байт по байт, в ред от най-малко значимия. Така байтовете по мрежата не зависят нито от подредбата на байтовете в машината, нито от подравняването на структурите, и едни и същи байтове минават през симулатора и през UDP дейтаграма. Общата част на пакета е осем байта: версия, вид, обща дължина и подател. Видовете са пет: Hello, DbDescription, LsRequest, LsUpdate и LsAck.

Декодирането използва курсор с проверка на границите. Всяко четене или премества курсора, или отбелязва курсора като негоден, и годността се проверява веднъж накрая вместо при всяко поле. Обявеният брой елементи се проверява отделно, преди да бъде използван за заделяне на памет: брой от над четири милиарда в пакет от четиридесет

байта е отхвърлен, преди векторът да поиска паметта. Пакет, чието поле за дължина не съвпада точно с получения буфер, се отхвърля като грешка в кадрирането. Четири от тестовите в набора `packet` проверяват точно тези откази, включително всяко възможно съкращаване на валиден пакет.

4.3. Автомат за състоянията на съседите

Съседството минава през последователност от състояния, в която двата възела установяват двупосочна видимост и след това синхронизират базите си, преди връзката да бъде обявена за използвана [2]. Автоматът е записан като чиста функция `neighbor_transition` от състояние и събитие към ново състояние и действие, а не като вложени условни оператори. Таблицата на преходите е дадена в табл. 1.

Таблица 1. Преходи на автомата за съседство

Състояние	Събитие	Ново състояние и действие
Down	HelloReceived	Init
Init	HelloReceived	Init
Init	TwoWayReceived	ExStart, изпраща обзор на базата
TwoWay	TwoWayReceived	ExStart, изпраща обзор на базата
ExStart	NegotiationDone	Exchange
Exchange	ExchangeDone	Loading, изпраща заявки
Loading	LoadingDone	Full, обявява връзката
всяко над Init	OneWayReceived	Init, отменя съседството
всяко	InactivityTimer	Down, отменя съседството
всяко	KillNeighbor	Down, отменя съседството

Всяка комбинация извън таблицата оставя състоянието непроменено и не предприема действие, което RFC 2328 нарича пренебрегнато събитие, а не грешка. Наборът `neighbor` съдържа девет теста, а последният от тях обхожда всичките петдесет и шест комбинации от седем състояния и осем събития и проверява, че функцията е тотална и че единствените преходи назад са трите определени пътя за разпадане.

Състоянието `ExStart` е мигновено, защото не се води преговор кой е водещ. RFC 2328 използва това състояние, за да определи водещ и подчинен и да въведе номерация на пакетите с обзор; тук обзорът е един пакет и номерация не е нужна. Състоянието е запазено в автомата, за да остане таблицата съпоставима с нормативната, а не защото носи поведение.

4.4. Синхронизиране на базите

Двата края влизат в `ExStart` с разлика до един интервал за поддържане, защото всеки от тях преминава нататък, когато види себе си изброен в приветствието на другия. Обзорът на по-бързия край пристига, докато по-бавният е още в `Init`. Първата реализация запазваше този обзор и го възпроизвеждаше при влизане в `Exchange`. Измерването показва, че решението е погрешно: запазеният обзор описва базата от по-ранен момент, а всичко, което по-бързият край е научил междувременно, не влиза нито в обзора, нито в лавинното разпространение, защото то заобикаля съседи под `Exchange`. При решетъчна мрежа четири на четири това оставяше една връзка непоявила се в нито една топология.

Решението е поле в пакета с обзор, което го отбелязва като отговор. Обзор без този признак получава точно един отговор, съдържащ базата такава, каквато е в момента на отговора; обзор с признака не получава отговор. Асиметрията прекратява размяната след най-много четири пакета и същевременно гарантира, че полученият обзор е пресен. Отделно от това лавинното разпространение достига съседи от `Exchange` нагоре, а не само `Full`, както предписва RFC 2328.

4.5. Разпространение на обявленията

Обявлението за състояние на връзка носи подател, пореден номер, възраст и списък от съседи с цени на връзките. Приемането сравнява поредния номер с този на записаното обявление от същия подател. Сравнението не е просто $>$: поредните номера се подреждат върху крайно множество, така че се взема знаковата разлика, $(\text{int32_t})(a - b) > 0$. Така 0 е по-ново от `0xFFFFFFFF`, тоест обръщането на брояча не спира приемането на новите обявления. Тестът `packet.sequence_numbers_compare_across_the_wrap` проверява точно това.

Изходът от приемането има пет случая, а не два, и разликата между тях определя реакцията:

- **Записано:** по-ново от притежаваното. Записва се, потвърждава се и се препраща по всички връзки без тази, по която е дошло.
- **Повторение:** същият пореден номер. Потвърждава се и не се препраща. Тази проверка е единственото, което спира лавината: без нея разпространението в пръстен не завършва.
- **По-старо:** притежаваме по-ново. В отговор се изпраща притежаваното, за да не чака

съседът следващото опресняване.

- **Оттеглено:** по-ново и с максимална възраст. Записът се изтрива и оттеглянето се препраща нататък.
- **Пренебрегнато:** максимална възраст за подател, който изобщо не е в базата. Не се препраща. Ако се препращаше, оттеглянето би обикаляло безкрайно, защото всеки следващ възел също няма какво да изтрие.

Възрастта не се увеличава от периодичен таймер. Всеки запис пази възрастта при получаване и момента на записване, а текущата възраст се пресмята при поискване. При виртуален часовник това е точно, а между събития не струва нищо.

Надеждността на разпространението се постига с потвърждения и повторно изпращане. Всяко изпратено обявление стои в списък за повторение до пристигане на потвърждение за същия подател и пореден номер. Заявките за липсващи обявления също се повтарят, но ограничен брой пъти: съсед, който е обявил нещо в обзора си и след това не може да го достави, го е оттеглил междувременно, и задържането на съседството заради това би струвало повече от липсващото обявление.

4.6. Изчисляване и инсталиране на маршрути

Изчисляването е алгоритъмът на Дийкстра [5] върху двупосочното подмножество на графа, с опашка с приоритет. Разстоянията се натрупват в 64 бита, за да не може верига от големи цени да препълни тихо 32-битовата метрика. Подредбата в опашката се разрешава по номер на възела при равни разстояния, така че обхождането не зависи от вътрешното устройство на контейнера.

Многопътността при равна цена се получава чрез сливане: при релаксация с равно разстояние към вече известен възел множествата от първи стъпки на двата пътя се обединяват, а предшественикът се добавя. Твърдението, което прави сливането коректно, е, че всички сливания към даден възел приключват преди той да бъде изваден от опашката, което е вярно, докато всички цени са строго положителни. Затова четецът на топологии отхвърля цена нула вместо да я приеме и да произведе тихо погрешна таблица.

Инсталирането е отделна стъпка от изчисляването и може да бъде забавено с параметър. Разделението е нужно, за да остане измерим интервалът между момента, в който възелът вече знае новата топология, и момента, в който тя влиза в сила.

Записването на изчислените маршрути в маршрутната таблица на операционната

система **не е реализирано**. Причината е, че то не влияе на измерваната величина: сходимостта се определя от таблиците на маршрутизаторите, а не от това дали ядрото на операционната система препраща по тях. Реализацията би изисквала три различни привилегировани интерфейса, `rtnetlink` на Linux, `PF_ROUTE` на macOS и `IP Helper` на Windows, всеки с различни изисквания за права, и би направила живия режим неизпълним без администраторски права. Записът остава в раздел 6.9 като посока за по-нататъшна работа.

4.7. Симулатор и внасяне на откази

Класът `Simulator` съдържа опашката, моделите на връзките, състоянието на отказите и дневника. Всеки маршрутизатор получава свой обект `NodeEnv`, който съдържа само указател към симулатора и собствения номер; цялото поведение е в симулатора, така че има само едно място, където виртуалното време се движи.

Възпроизводимостта се проверява, а не се твърди. Дневникът се свежда до 64-битова контролна сума по `FNV-1a` върху всичките шест полета на всяко събитие. Функцията е написана на място вместо да се вземе от библиотека, за да е една и съща на всяка платформа. Две изпълнения с еднакви входни данни трябва да дадат еднаква контролна сума; и програмата за демонстрация, и програмата за измерване правят тази проверка и отхвърлят резултата при несъвпадение.

4.8. Жив режим

Живият режим използва същия клас `Router`. Различава се само подадената изпълнителна среда: `UdpEnv` вместо `NodeEnv`. Слой за сокети е `UdpSocket` и събира трите разлики между `Winsock` и `BSD` сокетите: типа на манипулатора, извикването за затваряне и начина на съобщаване на грешка. Заглавните файлове на платформата се включват само в `src/udp.cpp`, така че `winsock2.h` не влиза в пътя на включване на нито един файл, който докосва протоколно състояние.

Тестът `udp.two_routers_form_an_adjacency_over_real_sockets` създава два маршрутизатора върху два реални сокета на обратната връзка и ги задвижва последователно, докато и двата достигнат `Full` и инсталират маршрут. Тестът `udp.a_datagram_from_a_stranger_is_discarded` изпраща обявление от подател, който не е настроен интерфейс, и проверява, че то не влиза в базата.

4.9. Съвместимост с еталонна реализация

Проверката за съвместимост с FRRouting или BIRD **не е проведена**. Причината е, че форматът по мрежата на Waypoint е собствен, а не форматът на OSPFv2 по RFC 2328. Еталонна реализация не би разпознала пакетите, така че проверката би измервала липсата на общ формат, а не съвпадението на изчислените маршрути. Да бъде проведена означава първо да се реализира кодирането по RFC 2328, което е отделна по обем задача. Вместо това коректността на изчислените маршрути се проверява срещу независим еталон вътре в системата: маршрутната таблица на всеки възел се сравнява с таблицата, изчислена от анализатора върху истинската топология. Разликата в силата на двете проверки е записана честно в раздел VI.

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА

5.1. Измервани величини

Измерват се четири величини.

Време за сходимост: интервалът от момента на отказа до момента, в който маршрутната таблица на всеки останал свързан възел съвпада точно с таблицата, изчислена независимо от анализатора върху истинската топология. Съпадението обхваща цената, множеството от следващи стъпки и множеството от предшественици, а не само достижимостта.

Преходни цикли: интервали, в които следващите стъпки на два или повече възела образуват затворен път за някоя двойка източник и цел. Отчита се броят на интервалите и общата им продължителност.

Натоварване от лавинното разпространение: брой съобщения и брой байтове, изпратени след отказа. Отчитат се отделно съобщенията с обявления и приветствията, защото приветствията са периодични и не зависят от събитието.

Нестабилност на маршрутните таблици: общ брой цели, за които някой възел е променил записа си след отказа.

Всички четири се извличат от маршрутните таблици и от дневника със събития след изпълнението.

5.2. Конфигурация на машината

Резултатите са получени при конфигурацията в табл. 2. Времето в симулатора е виртуално, така че тактовата честота, броят ядра и натовареността на машината **не влияят на измерените стойности**: те влияят единствено на това колко дълго тече самото измерване. Конфигурацията се записва, за да бъде възпроизводима компилацията, а не защото резултатът зависи от нея.

Таблица 2. Конфигурация, при която са получени резултатите

Операционна система	Windows 11 Pro N, версия 10.0.26200
Процесор	12 логически ядра
Компилатор	g++ 15.2.0, MinGW-w64, x86_64-posix-seh
Стандарт	C++20, -std=c++20 -Wall -Wextra -Wpedantic
Режим на изграждане	CMAKE_BUILD_TYPE=Release
Система за изграждане	CMake 4.3.2, генератор Ninja 1.13.2
Външни зависимости	няма

Пълното изпълнение на програмата за измерване, `waypoint-bench`, отнема около 8 секунди реално време на тази машина и произвежда всички числа в раздел VI.

5.3. Топологии

Използвани са четири семейства топологии, изброени в приложение Б. *Пръстенът* дава най-голям диаметър при даден брой възли и е следователно песимистичният случай. *Решетъчната мрежа* дава средна свързаност с алтернативни пътища. *Пълната мрежа* дава диаметър единица и максимално натоварване от лавинното разпространение. *Дебелото дърво* (fat tree) е топологията на съвременните центрове за данни и е включена, защото в нея пътищата с равна цена са правило, а не изключение.

Наборът за всеки експеримент е следният. За зависимостта от размера: пръстени с 4, 6, 8, 12, 16, 24 и 32 възела, тоест диаметър от 2 до 16. За зависимостта от таймерите: пръстен с 12 възела. За натоварването спрямо плътността: четири топологии с точно по 12 възела и средна степен от 2,00 до 11,00, за да бъде плътността единствената променлива. За вида на отказа: пръстен с 12 възела, решетъчна мрежа четири на четири и дебело дърво с k равно на четири.

Отказващата връзка е избрана по следния признак. В пръстен всички връзки са еквивалентни по симетрия, така че изборът не носи информация и не се нуждае от обосновка. В решетъчната мрежа и в дебелото дърво е избрана вътрешна връзка, защото отказ на периферна връзка засяга по-малко възли и би подценил измерваната величина.

5.4. Управлявани параметри

Стойностите по подразбиране са: интервал за поддържане 1 s, интервал за отпадане 4 s (отношението, препоръчано в RFC 2328 [2]), интервал за повторно изпращане 1

s, изчакване преди изчисление на маршрутите 100 ms, закъснение при инсталиране 0, закъснение на разпространение по връзка 1 ms и капацитет 100 Mbit/s. В експеримента за таймерите интервалът за поддържане приема стойности 250 ms, 500 ms, 1 s, 2 s и 4 s, а интервалът за отпадане е четирикратен на него.

В експеримента за преходните цикли на всеки маршрутизатор се задава закъснение при инсталиране, изтеглено веднъж от зърното равномерно в интервала от 0 до 200 ms. Основанието е, че записването на нова таблица за препращане отнема време, което зависи от апаратната част и от броя променени записи, така че два маршрутизатора не въвеждат новите си маршрути в един и същи момент. **Ако това закъснение е нула за всички, преходен цикъл е невъзможен по построение**, и измереният нула би бил свойство на модела, а не на протокола. Разсейването е част от условията на експеримента и се съобщава заедно с резултата.

5.5. Повторения и условия за валидност

Всяка комбинация се изпълнява по десет пъти със зърна от 1000 до 1009. Броят е избран така, че фазата между последното прието приветствие и момента на отказа да бъде извадена достатъчно на брой пъти: тази фаза е единственият източник на разсейване, тъй като времето е виртуално.

Всяко изпълнение има загряване от 40 s виртуално време, отказ на 41-ата секунда и прозорец за наблюдение от 60 s след отказа. Резултат се приема за валиден само при изпълнени три условия:

1. мрежата е достигнала сходимост преди отказа, иначе измерваното е опашката на началното установяване, а не възстановяване;
2. мрежата е достигнала сходимост отново в рамките на прозореца за наблюдение, иначе числото е долна граница, а не резултат;
3. повторното изпълнение със същото зърно дава **идентична контролна сума на дневника със събития**. Изпълнение, което не се повтаря, не е измерване.

Проверката по третото условие се прави за всяко изпълнение, тоест всеки резултат в раздел VI е получен от двойка изпълнения с проверено съвпадение.

5.6. Форма на резултатите

Резултатите се представят в четири таблици и една фигура. Таблиците дават времето за сходимост по размер на мрежата, времето за сходимост по стойност на таймерите, натоварването от лавинното разпространение по плътност и броя на преходните цикли по вид на отказа. Фигурата представя времето за сходимост като функция от диаметъра, с обхвата между най-малката и най-голямата стойност от повторенията.

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

6.1. Проведени измервания

Измерванията са проведени по методиката от раздел V с командата `waypoint-bench --csv results.csv`. Изпълнени са 25 комбинации по 10 повторения, тоест 250 резултата, всеки получен от двойка изпълнения с проверено съвпадение на контролната сума на дневника, или общо 500 изпълнения на симулатора.

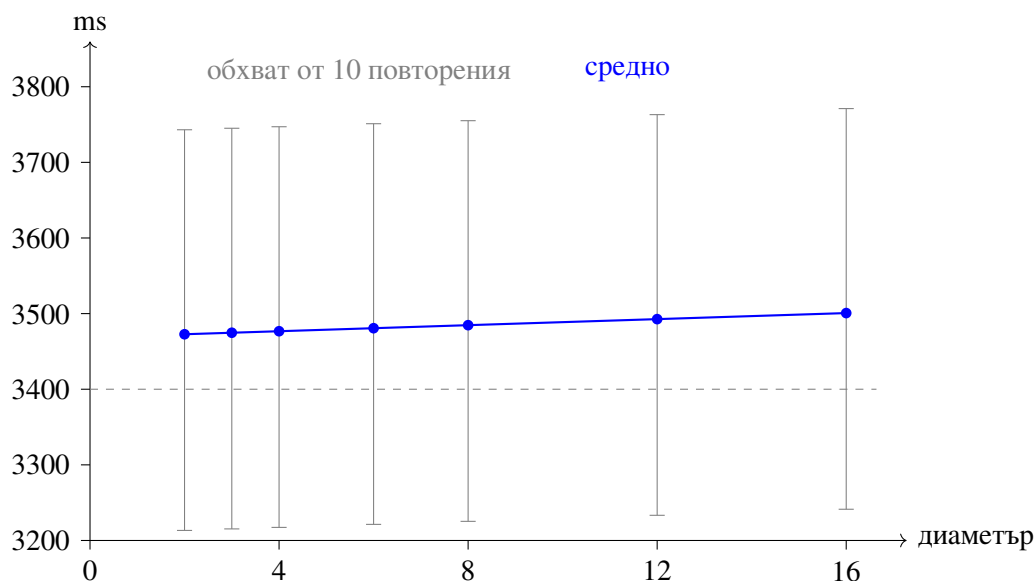
Отхвърлени изпълнения: 0. Всички 250 удовлетворяват и трите условия за валидност: мрежата е сходима преди отказа, сходима е отново в рамките на прозореца, и повторението със същото зърно дава идентична контролна сума. Отделно от това проверката за начална сходимост беше проведена върху 280 изпълнения на седем пръстена с по 40 различни зърна, а именно тестът `sim.cold_start_converges_on_every_ring_over_forty_seeds`, също без нито един отказ; най-бавното начално установяване при интервал за поддържане 1 s отне 2,098 s.

6.2. Време за сходимост спрямо размера на мрежата

Времето за сходимост след отказ на една връзка в пръстени с нарастващ диаметър е дадено (Табл. 3) и представено (Фиг. 2).

Таблица 3. Време за сходимост след отказ на връзка, пръстен, 10 повторения

Топология	Възли	Диаметър	Средно, ms	Най-малко, ms	Най-голямо, ms
ring4	4	2	3472,7	3213,3	3743,1
ring6	6	3	3474,7	3215,3	3745,1
ring8	8	4	3476,7	3217,3	3747,1
ring12	12	6	3480,7	3221,3	3751,1
ring16	16	8	3484,7	3225,3	3755,1
ring24	24	12	3492,7	3233,3	3763,1
ring32	32	16	3500,7	3241,3	3771,1



Фигура 2. Време за сходимост като функция от диаметъра на пръстена. Отсечките показват обхвата между най-малката и най-голямата стойност от десетте повторения.

Резултатът е, че **диаметърът почти не влияе**. Между диаметър 2 и диаметър 16 средното нараства с 28,0 ms, тоест с 2,0 ms на хоп, което съвпада точно със закъснението на разпространение по една връзка, зададено на 1 ms, умножено по двата хопа, добавяни към диаметъра от всеки следващ по големина пръстен. Същевременно разсейването между повторенията на една и съща топология е 529,8 ms, тоест **деветнадесет пъти по-голямо от целия ефект на диаметъра** в изследвания обхват.

Обяснението е в разпределението на времето. Откриването на отказа отнема почти целия интервал: то се извършва по таймера за отпадане, който е 4 s. Разпространението на обявлението през мрежата отнема милисекунди, а изчисляването на маршрутите е под измеримото. Затова диаметърът, който влияе само на разпространението, се вижда като наклон от 2 ms на хоп върху фон от около 3,5 s.

Практическото следствие е, че ускоряване на изчислението или на разпространението няма как да съкрати сходимостта осезаемо. Съкращаване се постига единствено чрез по-бързо откриване на отказа, което е и причината литературата да се съсредоточава върху механизми като двупосочното откриване на препращането [6].

6.3. Време за сходимост спрямо таймерите

Зависимостта от интервалите за поддържане и за отпадане е дадена (Табл. 4).

Таблица 4. Време за сходимост спрямо таймерите, пръстен с 12 възела, 10 повторения

Поддържане, s	Отпадане, s	Средно, ms	Най-малко	Най-голямо	Отношение
0,25	1	977,4	921,8	1033,2	0,98
0,50	2	1841,5	1721,3	2030,9	0,92
1,00	4	3480,7	3221,3	3751,1	0,87
2,00	8	6649,9	6221,3	7511,9	0,83
4,00	16	13007,7	12221,3	14351,1	0,81

Зависимостта е линейна и практически изцяло определена от интервала за отпадане. Средното време за сходимост е между 0,81 и 0,98 от него, като отношението намалява плавно с нарастване на интервала. Причината е, че отказът настъпва в момент, независим от фазата на приветствията: таймерът за отпадане на всеки край е бил презареден някъде в предходния интервал за поддържане, така че откриването отнема отпадането минус тази фаза. Колкото по-голям е интервалът за поддържане спрямо добавките от разпространение и изчисление, толкова по-голяма част от отношението изяжда фазата.

Наблюдаваният обхват потвърждава обяснението количествено: разликата между най-голямата и най-малката стойност е 111,4 ms при поддържане 250 ms, 529,8 ms при 1 s и 2129,8 ms при 4 s, тоест от 0,45 до 0,65 от интервала за поддържане във всичките пет случая. Разсейването е свойство на фазата, а не на топологията или на реализацията.

6.4. Натоварване от лавинното разпространение спрямо плътността

Натоварването при отказ на една връзка в четири топологии с точно по 12 възела и различна средна степен е дадено (Табл. 5).

Таблица 5. Натоварване след отказ на връзка, 12 възела във всички случаи, 10 повторения

Топология	Връзки	Степен	Съобщения	Байтове	Промени	Сходимост, ms
ring12	12	2,00	22,0	704	42,0	3480,7
mesh2x6	16	2,67	38,0	1520	54,0	3520,5
mesh3x4	17	2,83	42,0	2016	40,0	3421,5
full12	66	11,00	238,0	24752	2,0	3471,7

Броят на съобщенията расте приблизително пропорционално на броя връзки: от 12 на 66 връзки, тоест 5,5 пъти, съобщенията растат от 22 на 238, тоест 10,8 пъти.

Свърхпропорционалният растеж се обяснява с това, че всяко обявление се препраща по всяка връзка освен тази, по която е дошло, така че разходът за едно обявление е от порядъка на броя връзки, а броят на обявленията също расте с плътността.

Байтовете растат по-бързо от съобщенията: 35,2 пъти срещу 10,8 пъти. Причината е, че обявлението на един маршрутизатор изброява всичките му съседни, така че в пълна мрежа то съдържа 11 записа вместо 2 и е съответно по-дълго. Натоварването от лавинното разпространение расте следователно **квадратично със степента**, което е известното ограничение на протоколите по състояние на връзките и причината реалните разгръщания да делят мрежата на области.

Броят промени в маршрутните таблици върви в обратната посока: 42 при пръстена и 2 при пълната мрежа. В пълна мрежа отказът на една връзка променя маршрута само за двете двойки, които са я ползвали; в пръстен той променя посоката на препращане за половината от всички цели. Плътността тоест разменя натоварване от съобщения срещу устойчивост на маршрутните таблици.

Времето за сходимост не следва плътността: и четирите стойности са в диапазона от 3421 до 3521 ms, тоест в рамките на разсейването от фазата. Това е още едно потвърждение, че откриването на отказа, а не разпространението, определя сходимостта.

6.5. Преходни цикли спрямо вида на отказа

Броят и продължителността на преходните цикли при трите вида отказ и трите топологии са дадени (Табл. 6). Условието включват разсейване на закъснението при инсталиране от 0 до 200 ms на маршрутизатор, изтеглено от зърното.

Таблица 6. Преходни цикли по вид на отказа, 10 повторения, разсейване при инсталиране 200 ms

Топология	Вид отказ	Цикли	Продължителност, ms	Сходимост, ms
ring12	връзка, двупосочно	1,00	154,1	3710,2
ring12	връзка, едностранно	1,00	153,4	3862,7
ring12	възел	1,00	150,7	3858,3
mesh4x4	връзка, двупосочно	0,00	0,0	3654,6
mesh4x4	връзка, едностранно	0,00	0,0	3860,5
mesh4x4	възел	0,30	19,5	3858,9
fattree4	връзка, двупосочно	0,50	35,0	3605,1
fattree4	връзка, едностранно	0,50	35,4	3726,8
fattree4	възел	1,00	65,4	3725,7

Резултатът е, че **топологията определя дали изобщо възниква цикъл, а видът на отказа определя колко често**. В пръстен цикъл възниква при всяко от тридесетте изпълнения и при трите вида отказ. В решетъчна мрежа четири на четири отказ на връзка не произвежда цикъл в нито едно от двадесетте изпълнения, а отказ на възел произвежда в три от десетте.

Обяснението следва от геометрията на алтернативния път. В пръстен отказът на връзка кара възлите от отсрещната страна на прекъсването да препращат в обратна посока от преди. За интервала, в който единият край на дадено ребро вече е сменил посоката, а другият още не е, двата края сочат един към друг, което е точно преходен цикъл. В решетъчната мрежа алтернативният път заобикаля, но не обръща нито едно оцеляло ребро, така че такъв интервал не се появява. Отказът на възел в решетка засяга едновременно до четири връзки и вече може да обърне ребро, откъдето и ненулевата, но малка честота.

Едностранный отказ дава същия брой цикли като двупосочния, но по-бавна сходимост: 3862,7 срещу 3710,2 ms в пръстена и 3860,5 срещу 3654,6 ms в решетката, тоест от 153 до 206 ms повече. Причината е в пътя през автомата: при двупосочен отказ и двата края откриват мълчанието по своя таймер, докато при едностранен само единият край мълчи, а другият продължава да получава приветствия и трябва първо да види, че вече не е изброен в тях, преди да отмени съседството. Едната посока следователно преминава през допълнителен преход, а не отпада направо.

Продължителността на циклите е от порядъка на разсейването при инсталиране: 150,7 до 154,1 ms в пръстена при разсейване от 200 ms. Това е очакваното, тъй като циклите се затварят, когато и последният засегнат възел въведе новата си таблица.

6.6. Работен пример: пълна времева картина

Програмата `waupoint-demo` изпълнява един сценарий с подробна времева картина: пръстен с 12 възела, отказ на връзката между възли 6 и 7 на 40-ата секунда от виртуалното време, зърно 20260819. Резултатът е показан в табл. 7.

Таблица 7. Времева картина на едно възстановяване, всички моменти спрямо отказа

Възел	Научава, s	Изчислява, s	Инсталира, s
1	3,244	3,344	3,378
2	3,243	3,343	3,458
3	3,242	3,342	3,525
4	3,241	3,341	3,475
5	3,240	3,340	3,539
6	3,239	3,339	3,468
7	3,250	3,350	3,537
8	3,249	3,349	3,468
9	3,248	3,348	3,369
10	3,247	3,347	3,483
11	3,246	3,346	3,529
12	3,245	3,345	3,484

Картината показва трите фази поотделно. Възел 6 открива мълчанието пръв, на 3,239 s, и обявлението му обхожда целия пръстен за 11 ms: последният възел, номер 7, го получава на 3,250 s. Изчакването преди изчисление от 100 ms се вижда точно като постоянна разлика между втората и първата колона. Инсталирането се разстила от 3,369 до 3,539 s заради разсейването от 200 ms. Преходният цикъл, засегнал 24 двойки източник и цел, започва на 3,369 s, тоест в момента, в който първият възел въвежда новата си таблица, и се затваря на 3,539 s, когато последният прави същото. Мрежата е сходима на 3,539 s.

Разходът за възстановяването е 22 съобщения с обявления и 704 байта, при 42 променени цели общо и 24 изчисления на най-кратките пътища, тоест по две на възел. Изхвърлените по отказалата връзка пакети са 81, почти всички приветствия.

6.7. Възпроизводимост

Всички 500 изпълнения бяха повторени със същото зърно и дадоха идентична контролна сума на дневника със събития. За сценария от демонстрацията контролната сума е 956922bf7ea468f1 и при двете изпълнения. Условието за възпроизводимост, което е основното проектно твърдение на работата, се потвърждава без изключение.

6.8. Сверяване с еталонна реализация

Проверката за съвместимост с външна реализация на OSPF **не е проведена** по причината, изложена в раздел IV: форматът по мрежата на Waypoint е собствен и

еталонната реализация не би го разпознала. Коректността на изчислените маршрути е проверена срещу независим еталон вътре в системата: маршрутната таблица на всеки възел се сравнява с таблицата, изчислена отделно върху истинската топология, и това сравнение е самото условие за сходимост, така че то е изпълнено за всичките 250 валидни резултата.

Разликата в силата на двете проверки трябва да бъде отчетена: вътрешното сверяване проверява, че реализацията изчислява същото, което Дийкстра изчислява върху същия граф, но не проверява, че реализацията тълкува топологията така, както би я изтълкувал OSPF по RFC 2328. Второто остава непроверено.

6.9. Обсъждане

Три от очакванията, заложи в раздел V, се потвърждават, а едно се опровергава.

Потвърждава се, че сходимостта се определя от откриването на отказа: тя следва интервала за отпадане с отношение от 0,81 до 0,98 и не следва плътността. Потвърждава се, че натоварването от лавинното разпространение расте със свързаността, при това по-бързо от линейно по байтове. Потвърждава се, че преходните цикли са реални и измерими, с продължителност от порядъка на разсейването при инсталиране.

Опровергава се очакването, че времето за сходимост зависи съществено от диаметъра. То зависи, но с наклон от 2 ms на хоп, който в изследвания обхват е деветнадесет пъти по-малък от разсейването между повторенията. Очакването е било формулирано преди измерването и се оказва вярно по посока и несъществено по големина.

Ограниченията на измерването са четири. Първо, моделът на връзката няма загуби: пакет се губи само когато връзката е отказала, а не с някаква вероятност, така че поведението при влошена, но работеща връзка не е изследвано. Второ, разсейването при инсталиране е параметър на модела, а не измерена стойност от реална апаратура, така че абсолютната продължителност на циклите зависи от избора на този параметър; независим от него е фактът кои топологии изобщо произвеждат цикли. Трето, наборът от топологии е малък и правилен, а реалните мрежи не са. Четвърто, съвместимостта с външна реализация остава непроверена.

ЗАКЛЮЧЕНИЕ

Курсовият проект разглежда поведението на мрежа по стека TCP/IP в интервала между две установени състояния на маршрутизацията. Изградени са демон за маршрутизация по състояние на връзките и дискретно-събитиеен симулатор, който изпълнява същата протоколна логика върху управлявано време, така че отказът на връзка да бъде възпроизводимо събитие, а не еднократно наблюдение.

Основната проектна особеност на решението е, че протоколното ядро не притежава достъп до часовника и до сокетите, а работи над абстракция от пет операции. Това позволява едни и същи изходни файлове да бъдат изпълнявани както в симулация, така и над реални интерфейси, и прави измереното поведение свойство на реализацията, а не на неин отделен симулационен модел. Цената на решението е допълнителна граница между слоевете, която трябва да бъде поддържана и проверявана.

Получените резултати се свеждат до четири твърдения, всяко потвърдено от 250 валидни измервания и от нула отхвърлени.

Първо, времето за сходимост се определя почти изцяло от откриването на отказа. То следва интервала за отпадане с отношение от 0,81 до 0,98 и практически не зависи от диаметъра на мрежата: между диаметър 2 и диаметър 16 нараства с 2,0 ms на хоп, което е деветнадесет пъти по-малко от разсейването между отделните повторения на един и същ опит. Инженерното следствие е, че оптимизация на изчисляването или на разпространението не може да съкрати сходимостта; съкращаване се постига само чрез по-бързо откриване.

Второ, натоварването от лавинното разпространение расте по-бързо от свързаността. При преминаване от пръстен към пълна мрежа с еднакъв брой възли броят на връзките расте 5,5 пъти, броят на съобщенията 10,8 пъти, а броят на байтовете 35,2 пъти, защото по-плътната мрежа означава едновременно повече връзки и по-дълги обявления. Същевременно броят промени в маршрутните таблици намалява от 42 на 2. Плътността тоест разменя натоварване по мрежата срещу устойчивост на маршрутите.

Трето, преходните цикли са реални, измерими и зависят преди всичко от топологията, а не от вида на отказа. В пръстен цикъл се появи при всяко от тридесетте изпълнения, в решетъчна мрежа отказът на връзка не произведе цикъл в нито едно от двадесетте, а отказът на възел произведе в три от десет. Разграничителният признак е

дали алтернативният път обръща посоката на някое оцеляло ребро.

Четвърто, едностранният отказ на връзка се възстановява от 153 до 206 ms по-бавно от двупосочния, защото само единият край открива мълчание по таймера си, а другият трябва първо да установи, че вече не е изброен в приветствията на отсрещната страна.

Предимствата на решението, оценени спрямо измереното, са три. Възпроизводимостта е пълна: всичките 500 изпълнения дадоха идентична контролна сума на дневника при повторение, без нито едно изключение, което прави всяко от числата по-горе проверимо, а не само съобщено. Абстракцията се оказа достатъчна: същият клас Router работи и над виртуален часовник, и над реални UDP сокети, без нито един условен оператор по режим в протоколния код. Липсата на външни зависимости означава, че цялата верига от изграждане до измерване се възпроизвежда с компилатор и CMake.

Недостатъците, също спрямо измереното, са четири. Моделът на връзката няма загуби, така че поведението при влошена, но работеща връзка остава неизследвано. Разсейването при инсталиране на маршрутите е параметър на модела, а не измерена стойност от реална апаратура, така че абсолютната продължителност на циклите зависи от избора му, макар кои топологии изобщо произвеждат цикли да не зависи. Записването на изчислените маршрути в маршрутната таблица на операционната система не е реализирано. Съвместимостта с външна реализация на OSPF не е проверена, защото форматът по мрежата е собствен, а не този по RFC 2328.

Работата разкри и два дефекта, които заслужават да бъдат отбелязани, защото и двата бяха намерени от измерване, а не от разсъждение. Първият беше запазен обзор на базата, който при възпроизвеждане описваше по-ранно състояние и оставяше една връзка невидима за цялата мрежа. Вторият беше заявка за обявление, чийто отговор пристига по-стар от вече наученото от другаде и затова не изчистваше чакането, оставяйки съседството в междинно състояние завинаги. И двата се проявяваха само при определени зърна и щяха да останат незабелязани при изпълнение, което не е повторимо. Това е практическо потвърждение на изходната теза на проекта.

Като посоки за по-нататъшна работа се очертават: кодиране по формата на RFC 2328, което би направило възможно сверяването с FRRouting или BIRD; записване на маршрутите в маршрутната таблица на операционната система през `rtnetlink`, `PF_ROUTE` и `IP Helper`; модел на канал със загуби; включване на механизъм за по-бързо откриване на отказ като този в [6], чието влияние според резултатите тук би било решаващо; и свързване

на измерената сходимост с наблюдавана пропускателна способност на транспортния слой, което би позволило ефектът от преходните цикли да бъде оценен и откъм приложението.

ЛІТЕРАТУРА

- [1] C. Hedrick, “Routing information protocol,” RFC 1058, Internet Engineering Task Force, June 1988.
- [2] J. Moy, “OSPF version 2,” RFC 2328, Internet Engineering Task Force, April 1998. STD 54.
- [3] R. Coltun, D. Ferguson, J. Moy, and A. Lindem, “OSPF for IPv6,” RFC 5340, Internet Engineering Task Force, July 2008.
- [4] R. Perlman, *Interconnections: Bridges, Routers, Switches, and Internetworking Protocols*. Addison-Wesley, 2 ed., 2000.
- [5] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [6] D. Katz and D. Ward, “Bidirectional forwarding detection (BFD),” RFC 5880, Internet Engineering Task Force, June 2010.
- [7] V. Jacobson, “Congestion avoidance and control,” *ACM SIGCOMM Computer Communication Review*, vol. 18, no. 4, pp. 314–329, 1988.
- [8] M. Allman, V. Paxson, and E. Blanton, “TCP congestion control,” RFC 5681, Internet Engineering Task Force, September 2009.
- [9] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*. Pearson, 8 ed., 2021.
- [10] W. Eddy, “Transmission control protocol (TCP),” RFC 9293, Internet Engineering Task Force, August 2022. STD 7.
- [11] A. Varga and R. Hornig, “An overview of the OMNeT++ simulation environment,” in *Proceedings of the 1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems (SIMUTools)*, 2008.
- [12] G. F. Riley and T. R. Henderson, “The ns-3 network simulator,” in *Modeling and Tools for Network Simulation*, pp. 15–34, Springer, 2010.
- [13] ISO/IEC, “ISO/IEC 14882:2020 — programming languages — C++,” 2020.

ПРИЛОЖЕНИЯ

Приложение А. Формати на структурите от данни

А.1 Обща част на пакета

Всички цели числа са кодирани в ред от най-малко значимия байт, явно, байт по байт. Общата част е осем байта и предшества всеки от петте вида пакет.

Таблица 8. Обща част на пакета, 8 байта

Отместване	Размер	Поле	Значение
0	1	версия	винаги 1
1	1	вид	1 приветствие, 2 обзор, 3 заявка, 4 обявление, 5 потвърждение
2	2	дължина	дължина на целия пакет в байтове
4	4	подател	номер на изпращащия маршрутизатор

А.2 Приветствие и обявление за състояние на връзка

Таблица 9. Тяло на приветствието и на обявлението

Размер	Поле	Значение
Приветствие		
4	интервал за поддържане	в милисекунди
4	интервал за отпадане	в милисекунди
4	брой	брой на изброените съседи
4 <i>n</i>	съсед	маршрутизаторите, от които подателят е чувал скоро
Обявление, 12 + 8<i>m</i> байта		
4	подател	маршрутизаторът, чиито връзки се описват
4	пореден номер	поддържа се със знакова разлика, вж. раздел IV
2	възраст	в секунди; 3600 означава оттегляне
2	брой връзки	<i>m</i>
8 <i>m</i>	връзки	двойки от номер на съсед и цена, по 4 байта

Обзорът на базата носи признак за отговор (4 байта), брой (4 байта) и двойки от подател и пореден номер по 8 байта. Заявката носи брой и списък от номера на податели по 4 байта. Обявлението за актуализация носи брой и списък от обявления. Потвърждението има същото тяло като обзора без признака за отговор.

А.3 Запис в дневника със събития

Записът е с постоянна дължина и шест полета. Значението на последните две зависи от вида.

Таблица 10. Запис в дневника със събития

Поле	Тип	Значение
време	64 бита със знак	микросекунди, виртуални или монотонни
възел	32 бита	кой маршрутизатор е записал събитието
вид	8 бита	десетте вида, изброени по-долу
отсрещен	32 бита	съсед или подател на обявление
a, b	по 32 бита	зависят от вида

Видовете са: промяна на състоянието на съсед (a предишно, b ново състояние), породено обявление (a пореден номер, b брой връзки), прието обявление (същото), препратено обявление (a брой съседи, b пореден номер), изчислени маршрути (a пореден номер на изчислението, b брой цели), инсталирани маршрути (a брой променени цели, b размер на таблицата), отказ на връзка (a е 1 при едностранен отказ), възстановяване на връзка, отказ на възел и възстановяване на възел.

Приложение Б. Описание на топологиите

Топологиите се задават с текстов файл. Празните редове и всичко след знака # се пренебрегват. Директивата name задава име, node обявява възел, а link обявява връзка с незадължителни атрибути cost (цена), delay (закъснение в микросекунди) и bw (капацитет в битовете в секунда). Краищата на връзка стават възли и без изрична директива node. Ред, който не се разчита, се отхвърля с номера си; файл не се зарежда наполовина.

```

1 # Three routers in a line, with an expensive direct link between the ends.
2 name line3
3 link 1 2 cost 10 delay 1000 bw 100000000

```

```

4 link 2 3 cost 10 delay 1000 bw 1000000000
5 link 1 3 cost 50 delay 1000 bw 1000000000

```

Листинг 1: Топология с три възела, използвана от живия режим

Генераторите произвеждат четири семейства. `ring(n)` дава пръстен с n възела, n връзки и диаметър $\lfloor n/2 \rfloor$. `grid_mesh(r, c)` дава решетка с rc възела и $r(c-1) + (r-1)c$ връзки. `full_mesh(n)` дава пълна мрежа с $n(n-1)/2$ връзки и диаметър 1. `fat_tree(k)` за четно k дава $(k/2)^2$ централни комутатора и k групи по $k/2$ агрегиращи и $k/2$ крайни комутатора, тоест $5k^2/4$ възела и диаметър 4.

Използваните в раздел VI топологии са: `ring4` до `ring32`, `mesh2x6` и `mesh3x4` и `mesh4x4`, `full12` и `fattree4`.

Приложение В. Изходен текст

В.1 Автомат за състоянията на съседите

Автоматът е чиста функция от състояние и събитие. Двете събития, които се обработват във всяко състояние, са изнесени преди таблицата, точно както са изнесени и в RFC 2328.

```

1 Transition neighbor_transition(NeighborState state, NeighborEvent event) {
2     using S = NeighborState;
3     using E = NeighborEvent;
4     using A = NeighborAction;
5
6     // Two events apply in every state and are handled before the table,
7     ↪ exactly
8     // as RFC 2328 handles them.
9     if (event == E::KillNeighbor || event == E::InactivityTimer) {
10         if (state == S::Down) return Transition{S::Down, A::None};
11         return Transition{S::Down, A::ClearAdjacency};
12     }
13     if (event == E::OneWayReceived) {
14         // The far end stopped listing us. Everything above Init is void; the
15         // hearing itself survives, so we fall back to Init and not to Down.
16         if (state <= S::Init) return Transition{state, A::None};
17         return Transition{S::Init, A::ClearAdjacency};
18     }
19 }

```

```

17     }
18
19     switch (state) {
20         case S::Down:
21             if (event == E::HelloReceived) return Transition{S::Init, A::None};
22             break;
23
24         case S::Init:
25             if (event == E::HelloReceived) return Transition{S::Init, A::None};
26             if (event == E::TwoWayReceived) {
27                 return Transition{S::ExStart, A::SendDbDescription};
28             }
29             break;
30
31         case S::TwoWay:
32             // Reached only when an adjacency is torn down while both ends still
33             ↪ hear
34             // each other. The next confirmed hello restarts the exchange.
35             if (event == E::TwoWayReceived) {
36                 return Transition{S::ExStart, A::SendDbDescription};
37             }
38             break;
39
40         case S::ExStart:
41             if (event == E::NegotiationDone) return Transition{S::Exchange, A::None
42             ↪ };
43             break;
44
45         case S::Exchange:
46             if (event == E::ExchangeDone) return Transition{S::Loading, A::
47             ↪ SendRequests};
48             break;
49
50         case S::Loading:
51             if (event == E::LoadingDone) return Transition{S::Full, A::AdjacencyUp
52             ↪ };
53             break;
54
55         case S::Full:

```

```

52     break;
53 }
54 return Transition{state, A::None};
55 }

```

Листинг 2: Таблицата на преходите, `src/neighbor.cpp`

В.2 Изчисляване на най-кратките пътища с многопътност при равна цена

Сливането при равно разстояние е това, което произвежда множеството от следващи стъпки. Коректността му почива на строго положителните цени, което четецът на топологии налага.

```

1 RoutingTable shortest_paths(const Graph& graph, NodeId root) {
2     RoutingTable table;
3     if (graph.adj.find(root) == graph.adj.end()) return table;
4
5     // Distances are accumulated in 64 bits so that a chain of large link costs
6     // cannot silently wrap the 32 bit metric.
7     std::unordered_map<NodeId, std::uint64_t> dist;
8     std::unordered_map<NodeId, std::vector<NodeId>> next_hops;
9     std::unordered_map<NodeId, std::vector<NodeId>> parents;
10
11     using Item = std::pair<std::uint64_t, NodeId>;
12     // Ties break on node id, so the traversal order does not depend on the
13     // container's internal layout. Reproducibility starts here.
14     std::priority_queue<Item, std::vector<Item>, std::greater<Item>> queue;
15
16     dist[root] = 0;
17     queue.push({0, root});
18
19     while (!queue.empty()) {
20         const auto [d, u] = queue.top();
21         queue.pop();
22         const auto du = dist.find(u);
23         if (du == dist.end() || d > du->second) continue; // stale entry
24

```

```

25     const auto adj_it = graph.adj.find(u);
26     if (adj_it == graph.adj.end()) continue;
27
28     for (const Adjacency& edge : adj_it->second) {
29         const NodeId v = edge.peer;
30         if (v == root) continue;
31         const std::uint64_t nd = d + edge.cost;
32         const auto dv = dist.find(v);
33         if (dv == dist.end() || nd < dv->second) {
34             dist[v] = nd;
35             parents[v] = {u};
36             next_hops[v] = (u == root) ? std::vector<NodeId>{v} : next_hops[u];
37             queue.push({nd, v});
38         } else if (nd == dv->second) {
39             // Equal cost path: remember the extra parent and merge its first
40             ↪ hops.
41             insert_sorted_unique(parents[v], u);
42             if (u == root) {
43                 insert_sorted_unique(next_hops[v], v);
44             } else {
45                 merge_sorted_unique(next_hops[v], next_hops[u]);
46             }
47         }
48     }
49
50     for (const auto& [node, d] : dist) {
51         if (node == root) continue;
52         Route r;
53         r.cost = static_cast<Cost>(std::min<std::uint64_t>(d, kInfCost));
54         r.next_hops = next_hops[node];
55         r.parents = parents[node];
56         std::sort(r.parents.begin(), r.parents.end());
57         table.emplace(node, std::move(r));
58     }
59     return table;
60 }

```

Листинг 3: Алгоритъмът на Дийкстра с равноценни пътища, src/spf.cpp

В.3 Абстракцията върху времето и входа и изхода

Границата, която прави един и същ протоколен код изпълним и в симулатора, и над реални сокети, е пет метода.

```
1 class Env {
2     public:
3         virtual ~Env() = default;
4
5         // Current time in microseconds. Virtual under the simulator, monotonic
6         // under the daemon.
7         virtual Micros now() const = 0;
8
9         // One shot timer. There is deliberately no cancel operation: callbacks
10        ↪ that
11        // have been superseded compare a generation stamp and return. That keeps
12        ↪ the
13        // interface small and removes a whole class of dangling handle.
14        virtual void schedule(Micros delay, std::function<void()> fn) = 0;
15
16        // Hands a complete datagram to the layer below. Whether it becomes a
17        ↪ queued
18        // simulator event or a 'sendto' call is not the core's business.
19        virtual void send(NodeId self, NodeId peer, const Bytes& datagram) = 0;
20
21        // Seeded under the simulator so that timer jitter is reproducible.
22        virtual std::uint32_t random(std::uint32_t bound) = 0;
23
24        virtual void log(const LogEvent& event) { (void)event; }
25    };
26
```

Листинг 4: Интерфейсът Env, include/waypoint/env.hpp

В.4 Приемане на обявление

Петте изхода от записването в базата и различната реакция към всеки от тях.

```
1 void Router::handle_ls_update(NodeId from, ByteView datagram) {
2     const auto update = decode_ls_update(datagram);
3 }
4
```

```

3   if (!update) {
4       ++malformed_;
5       return;
6   }
7   const auto it = neighbors_.find(from);
8   if (it == neighbors_.end()) return;
9
10  LsAck ack;
11  bool database_changed = false;
12
13  for (const Lsa& lsa : update->lsas) {
14      ack.acked.push_back(LsaHandle{lsa.origin, lsa.seq});
15
16      if (lsa.origin == config_.id) {
17          // Our own advertisement came back with a sequence number ahead of ours
18          ↪ ,
19          // which happens after a restart. Jump past it and re-originate, or the
20          // network keeps believing a version of us that no longer exists.
21          if (seq_newer(lsa.seq, own_seq_)) {
22              own_seq_ = lsa.seq;
23              originate_lsa();
24          }
25          continue;
26      }
27
28      const Lsdb::Result result = lsdb_.install(lsa, env_.now());
29      // Whatever the database made of it, the request for this origin has been
30      // answered. Leaving it outstanding when the copy that arrived turned out
31      // to be older than one already learned elsewhere would hold the
32      ↪ adjacency
33      // in Loading for the rest of the run.
34      it->second.pending.erase(lsa.origin);
35      switch (result) {
36          case Lsdb::Result::Installed:
37              database_changed = true;
38              emit(EventKind::LsaInstalled, lsa.origin, lsa.seq,
39                  static_cast<std::uint32_t>(lsa.links.size()));
40              flood(lsa, from);
41              break;

```

```

40     case Lsdb::Result::Expired:
41         database_changed = true;
42         flood(lsa, from);
43         break;
44     case Lsdb::Result::Duplicate:
45         // Already held. Acknowledge and stop: this is the duplicate
46         // suppression that keeps flooding from circulating for ever.
47         it->second.rxmt.erase(lsa.origin);
48         break;
49     case Lsdb::Result::Older: {
50         // We hold something newer. Send it straight back rather than
↪ dropping
51         // silently, so the neighbour converges without waiting for a refresh
↪ .
52         if (auto newer = lsdb_.get(lsa.origin, env_.now())) {
53             LsUpdate reply;
54             reply.lsas.push_back(std::move(*newer));
55             env_.send(config_.id, from, encode_ls_update(config_.id, reply));
56         }
57         break;
58     }
59     case Lsdb::Result::Ignored:
60         break;
61 }
62 }
63
64 if (!ack.acked.empty()) {
65     env_.send(config_.id, from, encode_ls_ack(config_.id, ack));
66 }
67 if (database_changed) schedule_spf();
68 if (it->second.state == NeighborState::Loading && it->second.pending.empty
↪ ()) {
69     apply(from, NeighborEvent::LoadingDone);
70 }
71 }

```

Листинг 5: Обработка на пристигнало обявление, src/router.cpp

Приложение Г. Пълни резултати от измерванията

Програмата за измерване записва всички резултати в машинно четим вид с ключа `--csv`. Колоните са: експеримент, случай, брой възли, брой връзки, диаметър, средна степен, интервал за поддържане в милисекунди, интервал за отпадане в милисекунди, брой валидни, брой отхвърлени, средно, най-малко и най-голямо време за сходимост в милисекунди, брой съобщения с обявления, брой байтове, брой промени в маршрутните таблици, брой преходни цикли и обща продължителност на циклите в милисекунди.

Всички 250 резултата са валидни и нито един не е отхвърлен, така че таблиците в раздел VI съдържат пълния набор. Колоната за отхвърлени е нула на всеки ред.

Пълното изпълнение се възпроизвежда с:

```
1 cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
2 cmake --build build
3 ctest --test-dir build --output-on-failure
4 ./build/waypoint-bench --csv results.csv
5 cmake --build build --target demo
```

Листинг 6: Възпроизвеждане на всички резултати